GPUMODE Lecture Study Notes: Lecture 56
CUDA Kernel Benchmarking

Core Problem the Lecture Addresses

The lecture addresses a deceptively difficult engineering question:

How fast is my CUDA kernel?

At first glance, this seems like a simple timing problem. One might expect to put a timer before and after a kernel call, compute a duration, and report that as the kernel runtime. The lecture shows why that is usually wrong for CUDA programs.

The core problem is that CUDA kernel benchmarking sits at the boundary of three systems:

- **The CPU-side launch system**, which submits asynchronous work to the GPU.
- **The GPU execution system**, where kernels actually run on SMs.
- **The hardware power and thermal management system**, which changes GPU frequency dynamically.

A correct benchmark must isolate the GPU work being measured, avoid measuring unrelated CPU overheads, avoid first-run artifacts such as lazy module loading, avoid unrealistic cache states, account for thermal throttling, and summarize a noisy runtime distribution statistically.

The speaker distinguishes between two kinds of tools:

- **Profilers**: performance debuggers for one specific execution.
- **Benchmarks**: performance unit tests over many workloads and many measurements.

A profiler such as Nsight Systems or Nsight Compute answers:

What happened in this specific execution?

A benchmark answers:

How does this kernel behave over a workload space and runtime distribution?

The lecture uses `cub::DeviceSelect::If` and `DeviceHistogram` as running examples. These algorithms expose many real benchmarking problems: asynchronous launches, module loading, L2 cache warming, launch latency, stream synchronization, algorithmic non-determinism, thermal throttling, clock dependence, atomic contention, and statistical stopping criteria.

> A benchmark is not merely a stopwatch. A good CUDA benchmark is a controlled experiment over GPU execution states.

Required Background

CUDA Kernels Are Asynchronous

A CUDA kernel launch from the host returns before the GPU finishes executing the kernel. In simplified form:

```
kernel<<<grid, block>>>(args);
```

This statement usually only enqueues work into a CUDA stream. It does not wait for the work to complete. Therefore, CPU timers around this call measure mostly launch overhead, not GPU execution time.

The incorrect timing pattern is:

```
auto t0 = std::chrono::high_resolution_clock::now();

kernel<<<grid, block>>>(args);

auto t1 = std::chrono::high_resolution_clock::now();
```

This measures approximately:

$$T_{\text{measured}} \approx T_{\text{CPU launch}}$$

rather than:

$$T_{\text{desired}} \approx T_{\text{GPU execution}}$$

For short kernels, the launch overhead may dominate the measurement. For multi-kernel algorithms such as `DeviceSelect`, measuring only launch time can make the result look unrealistically fast.

CUDA Streams

A CUDA stream is an ordered queue of operations. Work submitted to the same stream executes in issue order. Work in different streams may overlap depending on dependencies and hardware resources.

For benchmarking, streams matter because timing markers such as CUDA events are also stream operations. If an event is recorded in a stream, it becomes ordered with respect to the kernels in that stream.

A typical CUDA event timing pattern is:

```
cudaEventRecord(start, stream);

kernel<<<grid, block, 0, stream>>>(args);

cudaEventRecord(stop, stream);
cudaEventSynchronize(stop);

float ms = 0.0f;
cudaEventElapsedTime(&ms, start, stop);
```

The intended measurement is:

$$T_{\text{event}} = T_{\text{stop event}} - T_{\text{start event}}$$

where both timestamps are taken by the GPU-side event system.

Nsight Systems Versus Nsight Compute

Nsight Systems

Nsight Systems provides a system-wide timeline. It shows CPU activity, GPU kernels, memory copies, synchronization, NVTX ranges, and temporal relationships between operations.

It is useful for answering questions such as:

- Which kernels were launched?
- Did CPU work overlap with GPU work?
- Were there gaps between kernels?
- Did memory copies overlap with compute?
- How long did an algorithm-level region take?

For a multi-kernel algorithm, Nsight Systems can show the wall-clock GPU timeline span of the whole algorithm.

Nsight Compute

Nsight Compute provides detailed kernel-level performance analysis. It reports metrics such as occupancy, memory throughput, instruction mix, cache behavior, warp stalls, and source-level correlations.

It is useful for answering questions such as:

- Is the kernel memory-bound or compute-bound?
- What is the achieved memory bandwidth?
- What are the dominant warp stall reasons?
- How many registers are used?
- How much shared memory is used?
- Are memory accesses coalesced?

However, Nsight Compute measures kernels individually. For a multi-kernel algorithm, it may not include the small gaps between kernels that Nsight Systems includes in an algorithm-level timeline span.

Device Select as the Running Example

The lecture uses `cub::DeviceSelect::If`. Conceptually, the algorithm reads an input range, applies a predicate to each element, and compacts selected elements into an output range.

For example, given input:

$$[1, 4, 3]$$

and predicate:

$$p(x) = \begin{cases} \text{true}, & x \text{ is odd}, \\ \text{false}, & x \text{ is even}, \end{cases}$$

the selected output is:

$$[1, 3]$$

The algorithm performs parallel filtering and compaction. Internally, efficient parallel compaction often requires prefix sums, tile states, inter-block coordination, or decoupled look-back style mechanisms. Thus, it is not a trivial single-kernel transformation.

In the lecture, `DeviceSelect` consists of two kernels:

- A first kernel initializes tile states per thread block in global memory.
- A second kernel performs the actual data loading, predicate evaluation, and compaction.

This is important because algorithm-level runtime is not always equal to the sum of individual kernel runtimes reported by a kernel profiler.

Main Concepts

Benchmarking Starts After Correctness

The first question about a kernel is correctness:

Does the kernel produce correct results?

This is answered with unit tests.

Only after correctness is established should one ask:

How fast is the kernel?

This is answered with profilers and benchmarks.

The lecture emphasizes that correctness and performance need different experimental tools. Unit tests are usually deterministic and pass or fail. Performance measurements are noisy distributions and require statistical interpretation.

Profilers Are Performance Debuggers

A profiler gives a detailed view of a specific execution. It helps debug performance pathologies such as:

- Low occupancy.
- Poor memory coalescing.
- Excessive DRAM traffic.
- High register pressure.
- Shared memory bank conflicts.
- Warp divergence.
- Atomic contention.
- Launch gaps.
- Unintended synchronization.

A profiler is analogous to a debugger because it helps answer:

What exactly happened in this run?

Profilers are not necessarily designed to produce statistically representative performance distributions over many workloads.

Benchmarks Are Performance Unit Tests

A benchmark is closer to a unit test for performance. It asks how a kernel behaves over many parameter choices.

For `DeviceSelect`, relevant benchmark axes include:

- Number of input elements.
- Element type, such as `char`, `int`, `float`, or `double`.
- Predicate selectivity.
- Input entropy and data distribution.
- Output size.
- Cache state.
- Single invocation versus repeated batch invocation.

An application-level profiler answers:

How fast is this kernel in this exact application use case?

A benchmark answers:

How fast is this kernel across a controlled workload space?

This distinction is critical. A kernel may look excellent in one application trace but regress over other important parameter settings.

First Trap: Measuring an Asynchronous Launch

A naive CPU timer around `DeviceSelect` may report something like a few microseconds, which is unrealistically fast for the actual amount of GPU work. This happens because the CPU timer measures enqueue time.

Wrong model:

$$T_{\text{chrono}} \approx T_{\text{enqueue}}$$

Correct target:

$$T_{\text{target}} \approx T_{\text{GPU algorithm}}$$

Adding `cudaDeviceSynchronize()` after the operation forces the CPU to wait for completion:

```
cudaDeviceSynchronize();

auto t0 = std::chrono::high_resolution_clock::now();

cub::DeviceSelect::If(...);

cudaDeviceSynchronize();

auto t1 = std::chrono::high_resolution_clock::now();
```

This is better than the naive version, but it still includes CPU-GPU synchronization overhead.

The measured interval becomes approximately:

$$T_{\text{measured}} = T_{\text{GPU algorithm}} + T_{\text{CPU-GPU sync overhead}}$$

This is not ideal when the goal is GPU execution time.

Second Trap: Lazy CUDA Module Loading

Starting with CUDA Toolkit 12.3, CUDA uses lazy module loading by default. Instead of loading all device code at application startup, CUDA loads device code on first use.

This avoids large startup overheads for applications with many compiled kernels but only a few used kernels. However, it affects first-run benchmark timing.

Let:

$$T_{\text{first run}} = T_{\text{module load}} + T_{\text{kernel execution}}$$

and:

$$T_{\text{subsequent run}} = T_{\text{kernel execution}}$$

A first-run benchmark may accidentally include:

$$T_{\text{module load}}$$

which is not part of steady-state kernel execution.

One way to avoid this is to force eager module loading using the environment variable:

```
CUDA_MODULE_LOADING=EAGER
```

Another way is to perform a warm-up invocation before timing.

Third Trap: Warm-Up Pollutes the Cache State

A warm-up run solves lazy module loading, but it can create an unrealistic cache state. If the first invocation reads the same data as the measured invocation, the second invocation may benefit from L2 cache residency.

The second measurement may become:

$$T_{\text{warm measured}} = T_{\text{kernel with warm L2}}$$

whereas a realistic cold-cache invocation might be:

$$T_{\text{cold measured}} = T_{\text{kernel with realistic memory traffic}}$$

For memory-sensitive kernels, this difference can be large.

Therefore, a benchmark may need to flush L2 cache between warm-up and measurement. A simple conceptual method is to write enough unrelated data to global memory to evict prior data from L2.

Fourth Trap: CPU Synchronization Overhead

Using `cudaDeviceSynchronize()` after the kernel forces the CPU to wait until the GPU finishes. The timestamp after synchronization is not exactly at the end of GPU work. It occurs later, after the synchronization completes.

Let:

$$T_S = \text{time when GPU work finishes}$$

and:

$$T_1 = \text{time when CPU records the end timestamp}$$

Then:

$$T_1 - T_S = T_{\text{synchronization overhead}}$$

This overhead contaminates CPU-side timing. CUDA events reduce this problem because the timestamps are recorded in the stream on the GPU side.

Fifth Trap: Kernel Launch Latency

CUDA event timing reduces CPU synchronization overhead, but a start event recorded before kernel launch may still be separated from the actual GPU kernel start by CPU launch latency.

The stream order is:

start event → kernel launch reaches stream → kernel executes → stop event

If the start event executes before the kernel has actually been submitted deeply enough, the measured time can include a gap:

$$T_{\text{event measured}} = T_{\text{launch gap}} + T_{\text{GPU execution}}$$

The lecture describes a more accurate method: block the stream, record the start event, enqueue the kernels, record the stop event, and then unblock the stream. This brings the start event closer to the actual kernel execution region.

Conceptually:

block stream → record start → enqueue kernels → record stop → unblock stream

Because the stream is blocked, all queued operations wait. Once unblocked, the start event and kernels execute back-to-back on the GPU timeline.

Sixth Trap: A Runtime Is a Distribution

A benchmark invocation does not produce a single universal runtime. It produces one sample from a distribution.

Let:

$$X = \text{runtime of one kernel invocation}$$

Then a benchmark observes samples:

$$x_1, x_2, \ldots, x_n$$

The mean is:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

The variance is:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2$$

The relative standard deviation, often reported as noise, is:

$$\text{Noise} = \frac{s}{\bar{x}}$$

A single runtime value can be misleading because it hides the spread, tails, and modes of the distribution. Some algorithms have intrinsic performance non-determinism due to concurrency, atomics, scheduling, and input-dependent behavior.

> The fairy tale is pretending that one timing number fully represents the performance of a GPU algorithm.

Hardware-Level Explanation

GPU Frequency and Thermal Throttling

Modern GPUs dynamically adjust frequency based on power, temperature, workload, and hardware limits. If a workload heats the GPU, the GPU may reduce clock frequency to remain within safe operating limits.

Let:

$$f(t) = \text{GPU core frequency at time } t$$

and let runtime depend on frequency:

$$T = g(f)$$

For compute-heavy kernels, a simple first-order model is:

$$T \propto \frac{1}{f}$$

More explicitly:

$$T(f) \approx \frac{C}{f}$$

where C is the approximate number of cycles required.

However, real kernels do not always follow this model. Memory-bound kernels may have weak dependence on SM frequency. Atomic-heavy kernels may even improve when SM frequency is reduced because lower issue rate can reduce contention.

Why Excessive Benchmark Repetition Can Be Harmful

One tempting approach is to run a kernel many times, such as 100,000 invocations, and average the result. The lecture shows that this can heat the GPU and cause frequency drops.

Suppose the first few samples run at high frequency:

$$f_1 \approx f_{\text{boost}}$$

but later samples run under throttling:

$$f_i < f_{\text{boost}}$$

Then later measurements are slower not because the algorithm changed, but because the hardware operating point changed.

The sample distribution then mixes different regimes:

$$p(T) = p(T \mid f_{\text{high}})p(f_{\text{high}}) + p(T \mid f_{\text{low}})p(f_{\text{low}}) + \cdots$$

This makes a single average hard to interpret. The benchmark is no longer measuring one workload under one representative hardware regime.

Why Locking Maximum Clocks Does Not Solve Everything

One might try to lock the GPU to maximum clocks. The lecture explains why this does not fully solve the issue.

If the GPU heats up, it must still protect itself. It cannot indefinitely maintain an unsafe frequency. Even if clocks are requested at a high value, the device may still throttle.

Thus:

$$f_{\text{actual}}(t) \neq f_{\text{requested}}$$

under thermal or power constraints.

Locking high clocks can cause the GPU to heat faster, leading to earlier throttling. Therefore, maximum clock locking does not necessarily reduce variance.

Why Locking Low Clocks Can Mislead Kernel Comparisons

Locking lower clocks can reduce thermal variation, but it can make the benchmark less representative of real user behavior. Users usually run GPUs in boost behavior, not artificially low-frequency modes.

More importantly, the ranking between two kernel versions may depend on frequency.

Let two implementations have runtimes:

$$T_A(f), \qquad T_B(f)$$

A performance comparison asks whether:

$$T_A(f) < T_B(f)$$

But it is possible that:

$$T_A(f_{\text{low}}) < T_B(f_{\text{low}})$$

while:

$$T_A(f_{\text{high}}) > T_B(f_{\text{high}})$$

Thus, a change that appears to improve performance at low clock can regress at high clock, or vice versa.

This is especially important for atomic-heavy algorithms, where contention depends on how fast operations are issued relative to memory-system progress.

Atomic Contention and Nonlinear Clock Dependence

The lecture uses `DeviceHistogram` to show a surprising result: lowering GPU core frequency can make an atomic-heavy histogram faster.

A simplified model is:

$$\lambda_{\text{atomic}}(f) = \text{rate at which SMs issue atomic operations}$$

As SM frequency increases:

$$\lambda_{\text{atomic}}(f) \uparrow$$

If many atomics target contended memory locations, increasing issue rate can overload the memory or atomic subsystem. The effective service time may increase due to contention.

A simplified queueing intuition is:

$$T_{\text{atomic}} \approx \frac{N}{\mu - \lambda}$$

where:

- N is the number of atomic operations.
- λ is the arrival rate of atomic requests.
- μ is the service rate of the memory or atomic subsystem.

As λ approaches μ, contention grows sharply. Reducing SM frequency can reduce λ, lowering contention and improving runtime.

This explains why performance as a function of frequency can be nonlinear:

$$T(f) \neq \frac{C}{f}$$

and may even have local minima at intermediate frequencies.

Memory-Bound Kernels and Weak Frequency Dependence

For a memory-bound kernel such as adjacent difference, runtime is dominated by memory bandwidth rather than SM core frequency.

A simple memory-bound model is:

$$T_{\text{memory}} = \frac{B_{\text{bytes}}}{\text{Bandwidth}_{\text{effective}}}$$

If effective DRAM bandwidth does not change much with SM core frequency, then:

$$\frac{\partial T}{\partial f_{\text{SM}}} \approx 0$$

This explains why some simple memory-bound kernels show little sensitivity to GPU clock changes.

Input Data Values Can Affect Power and Throttling

The lecture gives a subtle example: two workloads may take the same code paths and write the same number of output elements, but differ in data values. If values are close to zero, many bits are zero. This can reduce switching activity, current, heat, and throttling.

At the circuit level, dynamic power is roughly:

$$P_{\text{dynamic}} = \alpha C V^2 f$$

where:

- α is the switching activity factor.
- C is effective capacitance.
- V is voltage.
- f is frequency.

Different input bit patterns can change α. Therefore, artificial benchmark inputs such as all zeros may produce unrealistic thermal behavior.

A realistic benchmark must use representative data distributions, not merely convenient constants.

Visual Concept

Draw a GPU timeline with three layers. The top layer shows CPU launch calls. The middle layer shows CUDA stream events and kernels. The bottom layer shows GPU frequency over time decreasing as repeated benchmark invocations heat the chip. Mark where naive timing, CUDA event timing, and blocked-stream event timing place their start and stop points.

Mathematical Reasoning

Benchmark Measurement as a Random Variable

A CUDA kernel runtime should be modeled as a random variable:

$$X = T_{\text{kernel}}$$

A benchmark collects samples:

$$\mathcal{S} = \{x_1, x_2, \ldots, x_n\}$$

Useful summary statistics include:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2}$$

$$\text{Relative noise} = \frac{s}{\bar{x}}$$

A robust benchmark should not report only $\bar{x}$. It should also expose sample count, variation, and preferably the distribution.

Throughput

For a memory-oriented algorithm, throughput can be computed as:

$$\text{Throughput} = \frac{\text{Bytes processed}}{\text{Time}}$$

If a device select operation reads N elements of size b bytes and writes M selected elements, a logical byte count is:

$$B_{\text{logical}} = Nb + Mb$$

If selectivity is r, then:

$$M = rN$$

so:

$$B_{\text{logical}} = Nb(1 + r)$$

The achieved logical throughput is:

$$\text{Throughput}_{\text{logical}} = \frac{Nb(1 + r)}{T}$$

This differs from hardware DRAM throughput, because the implementation may read and write auxiliary data structures such as tile states. Logical throughput measures algorithmic efficiency relative to an ideal amount of problem data movement.

Speed-of-Light Utilization

If the peak memory bandwidth is:

$$\mathrm{BW}_{\mathrm{peak}}$$

then bandwidth utilization is:

$$\mathrm{Utilization} = \frac{\mathrm{Throughput}_{\mathrm{logical}}}{\mathrm{BW}_{\mathrm{peak}}}$$

For example, if a benchmark reports:

$$\mathrm{Throughput}_{\mathrm{logical}} = 700 \text{ GB/s}$$

and:

$$\mathrm{BW}_{\mathrm{peak}} = 1000 \text{ GB/s}$$

then:

$$\mathrm{Utilization} = \frac{700}{1000} = 0.70$$

or:

$$70\%$$

This is a speed-of-light style analysis. It answers:

How close is the implementation to an ideal bandwidth limit?

Shannon Entropy as a Stopping Criterion

The lecture introduces Shannon entropy to decide how many benchmark measurements are enough.

For a discrete random variable X with events x and probabilities $p(x)$, Shannon entropy is:

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$$

Here:

- X represents the measured runtime bucket or observed performance state.
- $p(x)$ is the empirical probability of observing state x.
- $H(X)$ measures the amount of information needed to describe the next observation.

The dice analogy is useful. If a die has only produced the value 6, then:

$$p(6) = 1$$

and:

$$H(X) = -1 \cdot \log_2(1) = 0$$

There is no uncertainty in the empirical model.

If two outcomes, 2 and 6, have appeared equally often, then:

$$p(2) = \frac{1}{2}, \qquad p(6) = \frac{1}{2}$$

and:

$$H(X) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1$$

As new runtime states are discovered, entropy increases. As the benchmark repeatedly observes already-known states, entropy saturates.

Entropy Saturation

Let H_n be the entropy after n measurements:

$$H_n = H(X_1, X_2, \ldots, X_n)$$

The stopping idea is to monitor the slope of entropy over a window. If entropy no longer increases meaningfully, the sample has likely discovered the representative runtime states.

A linear fit over a recent window can be written:

$$H_i \approx ai + b$$

for measurements i in the window. If:

$$|a| < \epsilon$$

with sufficient confidence, the benchmark stops.

This can reduce the number of measurements dramatically. Instead of always taking 100,000 samples, the benchmark may stop after hundreds or thousands of samples once the distribution is sufficiently characterized.

Frequency Filtering for Throttling

The lecture suggests measuring average GPU frequency during each benchmark sample and discarding samples that occur under obvious throttling.

Let:

$$f_{\max} = \text{maximum supported GPU frequency}$$

and let:

$$f_i = \text{average frequency during sample } i$$

A threshold τ can define acceptable samples:

$$f_i \geq \tau f_{\max}$$

For example, if:

$$\tau = 0.75$$

then samples below 75% of peak frequency are discarded.

The filtered sample set is:

$$\mathcal{S}_{\text{kept}} = \{x_i : f_i \geq \tau f_{\max}\}$$

This approach does not try to remove all noise. It only removes clearly unrepresentative thermal-throttling regimes.

Code Walkthrough

Naive CPU Timing

The naive approach is:

```
auto t0 = std::chrono::high_resolution_clock::now();

cub::DeviceSelect::If(temp_storage,
                      temp_storage_bytes,
                      d_input,
                      d_output,
                      d_num_selected,
                      num_items,
                      predicate);

auto t1 = std::chrono::high_resolution_clock::now();

double us =
    std::chrono::duration<double, std::micro>(t1 - t0).count();
```

This is wrong for GPU execution time because `DeviceSelect::If` launches asynchronous GPU work. The CPU timestamp t_1 is taken soon after the work is enqueued, not after it finishes.

The measured duration is approximately:

$$T_{\text{measured}} \approx T_{\text{host enqueue}}$$

This can produce impossible-looking results.

Adding Device Synchronization

A more plausible timing pattern is:

```
cudaDeviceSynchronize();

auto t0 = std::chrono::high_resolution_clock::now();

cub::DeviceSelect::If(temp_storage,
                      temp_storage_bytes,
                      d_input,
                      d_output,
                      d_num_selected,
                      num_items,
                      predicate);

cudaDeviceSynchronize();
```

```
auto t1 = std::chrono::high_resolution_clock::now();

double us =
    std::chrono::duration<double, std::micro>(t1 - t0).count();
```

The first synchronization prevents unrelated previous asynchronous work from contaminating the measurement. The second synchronization waits for the measured algorithm to finish.

However, the measurement now includes CPU-GPU synchronization overhead:

$$T_{\text{measured}} = T_{\text{GPU work}} + T_{\text{sync overhead}}$$

This is better than measuring launch latency, but still not ideal.

Warm-Up to Remove Lazy Loading

A warm-up run forces device code loading and other first-use overheads to happen before the timed region.

```
cub::DeviceSelect::If(temp_storage,
                      temp_storage_bytes,
                      d_input,
                      d_output,
                      d_num_selected,
                      num_items,
                      predicate);

cudaDeviceSynchronize();

auto t0 = std::chrono::high_resolution_clock::now();

cub::DeviceSelect::If(temp_storage,
                      temp_storage_bytes,
                      d_input,
                      d_output,
                      d_num_selected,
                      num_items,
                      predicate);

cudaDeviceSynchronize();

auto t1 = std::chrono::high_resolution_clock::now();
```

This removes first-use module loading from the measured invocation.

But if the same data is used, the warm-up may leave input data in L2 cache. Then the second run may measure a cache-warm scenario rather than a production-like scenario.

Cache Flushing

A conceptual L2 flush operation writes a sufficiently large buffer to evict prior cache contents.

```
__global__ void flush_kernel(char* buffer, std::size_t n)
{
    std::size_t idx =
        blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < n)
    {
        buffer[idx] = static_cast<char>(idx);
    }
}
```

A benchmark might call:

```
flush_kernel<<<grid, block, 0, stream>>>(flush_buffer,
                                         flush_buffer_size);
```

The flush buffer should be large enough to evict relevant data from L2. The intent is not to benchmark the flush; the intent is to restore a realistic cache state before timing the target kernel.

CUDA Event Timing

CUDA events reduce CPU-side timing overhead.

```
cudaEvent_t start;
cudaEvent_t stop;

cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start, stream);

cub::DeviceSelect::If(temp_storage,
                      temp_storage_bytes,
                      d_input,
                      d_output,
                      d_num_selected,
                      num_items,
                      predicate,
```

```
                        stream);

cudaEventRecord(stop, stream);
cudaEventSynchronize(stop);

float ms = 0.0f;
cudaEventElapsedTime(&ms, start, stop);
```

This measures GPU stream elapsed time between two event records.

Important details:

- The kernel should be launched on the same stream as the events.
- The stop event must be synchronized before reading elapsed time.
- This avoids placing the end timestamp after a full CPU-GPU synchronization.

However, it can still include a launch-related gap before the kernel begins.

Blocked-Stream Timing

The lecture describes a more accurate approach: block the stream, enqueue timing events and kernels, then unblock the stream. The high-level pseudocode is:

```
warm_up();
flush_l2();

block_stream(stream);

cudaEventRecord(start, stream);

launch_algorithm_on_stream(stream);

cudaEventRecord(stop, stream);

unblock_stream(stream);

cudaEventSynchronize(stop);
cudaEventElapsedTime(&ms, start, stop);
```

The purpose is to make the start event and kernels execute close together on the GPU after the stream is unblocked. This reduces the effect of CPU launch latency on the measured interval.

The exact implementation of `block_stream` and `unblock_stream` is framework-specific. NVBench hides this complexity.

Minimal NVBench Benchmark

NVBench abstracts warm-up, CUDA events, cache flushing, stream blocking, throttle detection, stopping criteria, parameter sweeps, and reporting.

A representative benchmark structure is:

```
#include <nvbench/nvbench.cuh>
#include <cub/cub.cuh>

struct is_odd
{
    __host__ __device__
    bool operator()(int x) const
    {
        return (x & 1) != 0;
    }
};

void device_select_bench(nvbench::state& state)
{
    const std::int64_t num_items =
        state.get_int64("Elements");

    thrust::device_vector<int> input(num_items);
    thrust::device_vector<int> output(num_items);
    thrust::device_vector<int> num_selected(1);

    int* d_input = thrust::raw_pointer_cast(input.data());
    int* d_output = thrust::raw_pointer_cast(output.data());
    int* d_num_selected =
        thrust::raw_pointer_cast(num_selected.data());

    void* temp_storage = nullptr;
    std::size_t temp_storage_bytes = 0;

    cub::DeviceSelect::If(temp_storage,
                          temp_storage_bytes,
                          d_input,
                          d_output,
                          d_num_selected,
                          num_items,
                          is_odd{});

    thrust::device_vector<char> temp(temp_storage_bytes);
    temp_storage = thrust::raw_pointer_cast(temp.data());
```

```
    state.add_global_memory_reads<int>(num_items);
    state.add_global_memory_writes<int>(num_items);

    state.exec([&](nvbench::launch& launch)
    {
        cudaStream_t stream = launch.get_stream();

        cub::DeviceSelect::If(temp_storage,
                              temp_storage_bytes,
                              d_input,
                              d_output,
                              d_num_selected,
                              num_items,
                              is_odd{},
                              stream);
    });
}

NVBENCH_BENCH(device_select_bench)
    .add_int64_axis("Elements", {1 << 20, 1 << 24, 1 << 28});
```

The benchmark function receives an `nvbench::state`. The state provides the current workload parameters and exposes `state.exec`, which contains the region to benchmark.

The code before `state.exec` is setup and is not timed. This is where one should allocate memory, initialize input data, prepare temporary storage, and register logical memory traffic.

The lambda passed to `state.exec` is the measured launcher. It should enqueue GPU work onto the stream provided by NVBench:

```
cudaStream_t stream = launch.get_stream();
```

Using the provided stream is crucial because NVBench controls timing and stream blocking around that stream.

Parameter Sweeps

NVBench can sweep over workload axes.

```
NVBENCH_BENCH(device_select_bench)
    .add_int64_axis("Elements", {1 << 20, 1 << 24, 1 << 28});
```

This creates one benchmark workload for each value of `Elements`. If multiple axes are registered, NVBench evaluates the Cartesian product.

For example:

$$\mathcal{W} = \mathcal{N} \times \mathcal{T} \times \mathcal{R}$$

where:

- $\mathcal{N}$ is a set of problem sizes.
- $\mathcal{T}$ is a set of data types.
- $\mathcal{R}$ is a set of selectivity ratios.

This is exactly why benchmarks are performance unit tests. They cover many workloads, not just one application trace.

Disabling Batch Measurements

NVBench can report single-invocation GPU time and batch GPU time. If batch timing is not meaningful for a kernel, it can be disabled conceptually with an execution tag.

A representative pattern is:

```
state.exec(nvbench::exec_tag::no_batch,
           [&](nvbench::launch& launch)
{
    cudaStream_t stream = launch.get_stream();
    kernel<<<grid, block, 0, stream>>>(args);
});
```

Batch measurements are useful when production uses repeated back-to-back invocations or relies on L2 cache residency between calls. They are less useful when the kernel is normally used as an isolated operation.

Synchronous Launchers

NVBench stream blocking assumes that the measured launcher does not synchronize with the GPU. If the launcher calls a synchronizing API, a deadlock can occur because the stream is intentionally blocked.

Problematic APIs include:

- `cudaDeviceSynchronize`.
- `cudaStreamSynchronize`.
- `cudaEventSynchronize`.
- `cudaFree`, because it can have synchronizing behavior.

If synchronization is unavoidable, use a synchronous execution mode. Conceptually:

```
state.exec(nvbench::exec_tag::sync,
           [&](nvbench::launch& launch)
{
    cudaStream_t stream = launch.get_stream();

    kernel<<<grid, block, 0, stream>>>(args);

    cudaStreamSynchronize(stream);
});
```

This disables some accuracy mechanisms such as stream blocking. It may include launch overhead and reduce throttle detection quality. Therefore, it is better suited for larger workloads where launch overhead is small relative to kernel runtime.

CMake Integration

A representative CMake integration using CPM looks like:

```
CPMAddPackage(
  NAME nvbench
  GITHUB_REPOSITORY NVIDIA/nvbench
  GIT_TAG main
)

add_executable(my_benchmark benchmark.cu)
target_link_libraries(my_benchmark PRIVATE nvbench::main)
```

Compile benchmarks in release mode. Debug builds can disable important compiler optimizations and produce meaningless performance results.

A release build should use flags conceptually similar to:

```
cmake -DCMAKE_BUILD_TYPE=Release ..
cmake --build . -j
```

Do not benchmark unoptimized debug code unless the purpose is specifically to understand debug overhead.

Performance Intuition

Why Nsight Systems and Nsight Compute Can Disagree

Nsight Systems may report an algorithm-level NVTX range around multiple kernels. Nsight Compute reports individual kernel durations.

If an algorithm has two kernels, then:

$$T_{\text{algorithm}} = T_{K_1} + T_{\text{gap}} + T_{K_2}$$

Nsight Compute may report:

$$T_{\text{Ncu}} = T_{K_1} + T_{K_2}$$

while Nsight Systems may report:

$$T_{\text{Nsys}} = T_{K_1} + T_{\text{gap}} + T_{K_2}$$

The difference is:

$$T_{\text{Nsys}} - T_{\text{Ncu}} = T_{\text{gap}}$$

where T_{gap} includes setup and teardown between kernels.

Additionally, Nsight Compute may lock clocks by default for reproducibility. If the lock is at a base frequency such as 900 MHz while the GPU can boost much higher, measured runtime may be much slower than in normal operation.

Why Warm-Up Helps and Hurts

Warm-up helps because it removes one-time overheads:

$$T_{\text{first use}} = T_{\text{module loading}} + T_{\text{allocation side effects}} + T_{\text{kernel execution}}$$

But warm-up hurts if it changes cache state:

$$T_{\text{second run}} = T_{\text{kernel execution with warm cache}}$$

A correct benchmark needs both:

- Warm-up to remove first-use overheads.
- Cache flushing to avoid unrealistic L2 residency when needed.

Why CPU Clock Locking Usually Does Not Help

If the benchmark uses blocked-stream CUDA event timing, CPU launch overhead is mostly excluded from the measured interval. Therefore, CPU frequency variation has little effect.

The CPU still prepares launches, but launch latency is not the target metric. The GPU-side event interval is designed to isolate GPU work.

CPU clock locking becomes more relevant when:

- The benchmark uses synchronous launchers.
- Launch overhead is included in the measurement.
- The measured workload is extremely small.
- The benchmark includes CPU-side setup inside the timed region.

Single Invocation Versus Batch Timing

Some kernels are used once in isolation. Others are used in hot loops. These use cases can prefer different implementations.

For a single invocation, one might optimize:

$$T_{\text{single}}$$

For repeated invocations, one might optimize average batch time:

$$T_{\text{batch avg}} = \frac{1}{B} \sum_{i=1}^{B} T_i$$

where B is the number of back-to-back invocations.

Batch behavior can differ because:

- Data may remain in L2 cache.
- Thermal behavior may matter more.
- A less aggressive kernel may throttle less.
- Contention behavior may change over repeated calls.

The lecture gives a `DeviceSelect` example where adding delay may hurt isolated runtime but improve batch behavior by reducing memory contention and throttling.

Logical Bandwidth Versus Hardware Bandwidth

NVBench can report logical memory traffic based on user-declared reads and writes. Nsight tools can report hardware counters such as actual DRAM throughput.

These are different.

Logical traffic:

$$B_{\text{logical}} = \text{minimum problem data movement}$$

Hardware traffic:

$$B_{\text{hardware}} = \text{actual DRAM bytes moved}$$

Usually:

$$B_{\text{hardware}} \geq B_{\text{logical}}$$

because implementations may use auxiliary metadata, tile state arrays, extra loads, writebacks, or cache-line effects.

Logical throughput is useful for speed-of-light reasoning. Hardware throughput is useful for diagnosing implementation behavior.

Common Misconceptions

Misconception: One Timing Number Is Enough

A single runtime is only one sample. CUDA kernel performance is a distribution. A single run can be lucky or unlucky due to cache state, scheduling, thermal state, and concurrent algorithm behavior.

Correct view:

$$\text{Kernel performance} = \text{distribution of runtimes}$$

not:

$$\text{Kernel performance} = \text{one scalar}$$

Misconception: More Samples Always Means Better Benchmarks

More samples can heat the GPU and cause throttling. After enough repetitions, the benchmark may measure a thermally degraded state rather than representative production behavior.

The goal is not maximum sample count. The goal is enough samples to characterize the distribution without drifting into unrealistic hardware states.

Misconception: Locking GPU Clocks Solves Noise

Locking high clocks does not prevent thermal throttling. Locking low clocks can make comparisons unrepresentative and even reverse performance conclusions.

Correct view:

> Clock locking can change the experiment.

It is not a universal solution.

Misconception: Warm-Up Is Always Sufficient

Warm-up removes lazy loading and first-use overheads but can warm L2 cache. A benchmark that only warms up may measure an artificially favorable cache state.

Correct view:

> Warm-up must be paired with cache-state control.

Misconception: CPU Timers Measure GPU Work

CPU timers measure CPU-observed intervals. Without synchronization, they measure launch time. With synchronization, they include CPU-GPU synchronization overhead.

Correct view:

> Use CUDA events and stream-aware timing for GPU work.

Misconception: Artificial Inputs Are Fine

Inputs full of constants or zeros may not exercise realistic power, cache, branch, or memory behavior. Even if code paths are identical, bit patterns can affect switching activity, heat, and throttling.

Correct view:

> Benchmark data distribution is part of the workload definition.

Misconception: Benchmarks Replace Profilers

Benchmarks and profilers answer different questions. Benchmarks detect regressions and compare workloads. Profilers explain why a kernel behaves as it does.

Correct view:

Use benchmarks for performance testing and profilers for performance debugging.

Practical Takeaways

Benchmark Design Checklist

A serious CUDA kernel benchmark should consider:

1. Correctness is already tested.
2. The benchmark uses representative workload parameters.
3. The benchmark uses representative input distributions.
4. First-use overheads such as lazy module loading are removed.
5. Cache state is controlled or intentionally modeled.
6. Timing uses CUDA events rather than naive CPU timers.
7. CPU-GPU synchronization overhead is not included unless intentionally measured.
8. Kernel launch latency is excluded when measuring GPU execution time.
9. Multiple samples are collected.
10. Runtime distribution and noise are reported.
11. Thermal throttling is detected and unrepresentative samples are discarded.
12. Sample count is chosen statistically, not arbitrarily.
13. Benchmarks are compiled in release mode.
14. Synchronous APIs inside the measured launcher are avoided unless explicitly handled.

When to Use Nsight Systems

Use Nsight Systems when asking:

- What is the end-to-end application timeline?
- Are there gaps between kernels?
- Are kernels overlapping with memory copies?
- Is the CPU failing to feed the GPU?
- How long does an NVTX algorithm region take?

When to Use Nsight Compute

Use Nsight Compute when asking:

- Why is this kernel slow?
- Is it memory-bound or compute-bound?
- What is the achieved occupancy?
- What are the warp stall reasons?
- What is the L1 or L2 hit rate?
- Are atomics causing contention?
- Is shared memory used efficiently?

When to Use NVBench

Use NVBench when asking:

- Did my kernel regress?
- Which parameter setting is faster?
- How does performance vary across problem sizes?
- What is the runtime distribution?
- How noisy is the kernel?
- What is the logical memory throughput?
- Can this benchmark run in performance CI?

CI Implications

A performance CI system should not compare one runtime against another runtime blindly. It should compare statistically meaningful summaries.

A better CI comparison considers:

$$\bar{x}_{\text{old}}, \quad s_{\text{old}}, \quad \bar{x}_{\text{new}}, \quad s_{\text{new}}, \quad n_{\text{old}}, \quad n_{\text{new}}$$

A reported regression might be:

$$\text{Regression} = \frac{\bar{x}_{\text{new}} - \bar{x}_{\text{old}}}{\bar{x}_{\text{old}}}$$

If:

$$\text{Regression} = 0.04$$

then the new version is 4% slower on average. But the conclusion should be interpreted relative to noise. A 4% regression is meaningful if noise is 1%, but less convincing if noise is 10%.

Project or Experiment Ideas

Experiment 1: Naive Timer Versus CUDA Event Timer

Implement a simple vector add kernel and measure it using:

- CPU timer without synchronization.
- CPU timer with `cudaDeviceSynchronize`.
- CUDA events.

Compare:

$$T_{\text{naive}}, \qquad T_{\text{sync}}, \qquad T_{\text{event}}$$

Expected result: naive timing is too small, synchronized CPU timing includes overhead, and CUDA events better represent GPU work.

Experiment 2: Warm L2 Versus Flushed L2

Benchmark a memory-bound kernel twice:

- Once immediately after a warm-up using the same data.
- Once after flushing L2 with a large unrelated buffer.

Measure:

$$T_{\text{warm L2}} \qquad \text{and} \qquad T_{\text{flushed L2}}$$

Expected result: cache-sensitive kernels run faster with warm L2.

Experiment 3: Runtime Distribution

Run a kernel many times and plot the distribution of runtimes. Compute:

$$\bar{x}, \qquad s, \qquad \frac{s}{\bar{x}}$$

Check whether the distribution is unimodal or multimodal. If multimodal, investigate possible causes such as frequency changes, cache effects, or algorithmic non-determinism.

Experiment 4: Clock Sensitivity

Benchmark three kernels under different GPU clock regimes:

- A compute-heavy kernel.
- A memory-bound copy-like kernel.
- An atomic-heavy histogram kernel.

Study the shape of:

$$T(f)$$

Expected results:

- Compute-heavy kernels often improve with frequency.
- Memory-bound kernels may barely change.
- Atomic-heavy kernels may show nonlinear behavior.

Experiment 5: Input Bit Patterns and Throttling

Benchmark the same kernel using:

- All-zero input.
- Uniform random input.
- High-entropy random input.

Track runtime and GPU frequency. This tests whether data values affect power and throttling.

Experiment 6: NVBench Parameter Sweep

Write an NVBench benchmark for a CUDA reduction kernel. Sweep:

- Elements per thread.
- Threads per block.
- Input size.
- Data type.

Record which combinations produce the best throughput. Compare against Nsight Compute metrics to explain why.

Final Mental Model

CUDA kernel benchmarking is controlled measurement of GPU work under realistic hardware and workload conditions.

A profiler tells you what happened in one execution. A benchmark tells you how performance behaves over a workload space and over a runtime distribution.

The main traps are:

- CUDA launches are asynchronous.
- CPU timers do not naturally measure GPU execution.
- First runs include lazy module loading.
- Warm-up can produce unrealistic L2 cache residency.
- Synchronization overhead can contaminate timing.
- Kernel launch latency can leak into event intervals.
- GPU frequency changes during repeated runs.
- Thermal throttling can dominate large sample sets.
- Artificial inputs can change power behavior.
- Atomic-heavy kernels can respond nonlinearly to clock changes.
- One scalar cannot fully represent a noisy distribution.

The clean mental model is:

Good benchmark = representative workload+accurate GPU timing+controlled cache state+thermal awarenes

NVBench exists to automate much of this machinery: warm-up, CUDA event timing, stream blocking, cache flushing, throttle detection, entropy-based stopping, parameter sweeps, logical throughput reporting, JSON output, and profiler-compatible modes.

> Do not benchmark CUDA kernels as if they were ordinary CPU functions. Benchmark them as asynchronous, thermally dynamic, cache-sensitive, statistically noisy hardware experiments.